Project 1

Simple App — Learn to See the System

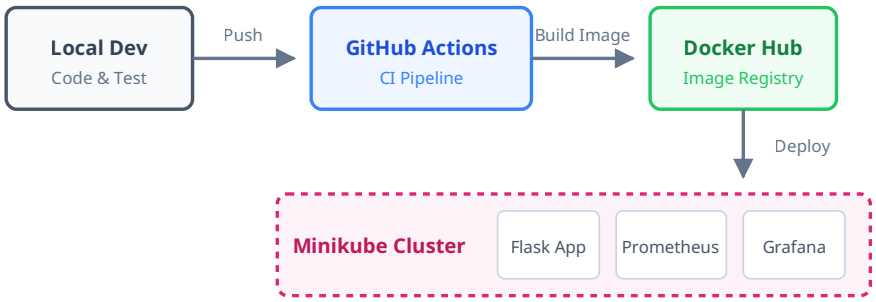
Estimated Duration: 3–4 Weeks

Primary Goal: The focus of this phase is *not* the application itself, but the ecosystem around it. You will build a tiny Python app and wire together a complete deployment and monitoring pipeline to understand how modern infrastructure tools interact.

Tech Stack & Tools

Python (Flask) · Docker · Minikube · GitHub Actions · Prometheus · Grafana · Linux

System Architecture Pipeline



Part 1: The "Happy Path" (Building the System)

In this section, your goal is to get the system running perfectly from end to end.

1. The Application Baseline

- Write a minimalist Python Flask application. It only needs one page and one route (e.g., returning "Hello, World!" and an HTTP 200 status).
- Run and test it locally on your Linux environment.

2. Containerization

- Write a `Dockerfile` for your Flask app.
- Build the Docker image and verify that the container runs successfully on your local machine.

3. Automation Pipeline

- Create a GitHub repository for your code.
- Set up a GitHub Actions workflow (YAML). Configure it so that every time you push code, the pipeline automatically lints the code, builds the Docker image, and pushes it to Docker Hub (or a local registry).

4. Local Orchestration

- Start a Minikube cluster.
- Write Kubernetes manifests (`deployment.yaml` and `service.yaml`) to deploy your Docker image into the Minikube cluster.
- Expose the service and verify you can access the Flask app through your browser.

5. Observability & Monitoring

- Deploy Prometheus and Grafana into your Minikube cluster.
- Configure Prometheus to scrape basic metrics from your cluster.
- Set up a simple Grafana dashboard to visualize CPU usage, memory consumption, and network traffic for your Flask app pod.

Part 2: Chaos & Troubleshooting

Systems break. As engineers, you need to know what failure looks like so you can fix it. Once your "Happy Path" is fully functional, you will intentionally sabotage your system and document the symptoms, logs, and recovery steps.

Task: Execute each of the following failure scenarios. For each scenario, document the **Error State**, the **Symptoms**, and the **Fix**.

Scenario A: Crash the Pod

Action: Use `kubectl exec` to enter your running pod and manually kill the Flask process.
Observation: Watch how Kubernetes handles the sudden termination of the main container process. Does it restart? How long does it take?

Scenario B: Set Wrong Memory Limits

Action: Edit your `deployment.yaml` to set an impossibly low memory limit (e.g., `5Mi`) for the container. Apply the changes.
Observation: Observe the pod failing to schedule or crashing. Look for the `OOMKilled` (Out of Memory) status in your `kubectl describe pod` output.

Scenario C: Push a Broken Docker Image

Action: Update your Kubernetes deployment to pull an image tag that doesn't exist (e.g., `my-flask-app:does-not-exist`).
Observation: Watch the pod state transition to `ImagePullBackOff` or `ErrImagePull`. Check the Kubernetes events log.

Scenario D: Break the Pipeline YAML

Action: Introduce a deliberate syntax error (like bad indentation) into your GitHub Actions `.yaml` file and commit the change.
Observation: Navigate to the "Actions" tab in GitHub. Observe the workflow failure before the build even begins.

Scenario E: Provide a Wrong Environment Variable

Action: Modify your Flask app to depend on a specific environment variable to start up, but intentionally leave it out or misspell it in your Kubernetes `deployment.yaml`.

Observation: The pod will likely enter a `CrashLoopBackOff`. Use `kubectl logs` to find the Python traceback showing the missing environment variable.

Scenario F: Kill a Node

Action: While the application is running, forcibly stop the Minikube instance (simulate a sudden server failure).

Observation: Restart Minikube and observe how Kubernetes attempts to restore the desired state of your deployment automatically.

Deliverables & Submission

At the end of this phase, you must submit:

1. **Repository Link:** A link to your GitHub repo containing the app code, Dockerfile, GitHub Actions workflow, and Kubernetes manifests.
2. **System Proof:** Screenshots showing the app running in Minikube and your active Grafana dashboard.
3. **Troubleshooting Log:** A short report detailing the logs, symptoms, and fixes for all 6 intentional failure scenarios.